

RTCM — Runtime Trust Continuity Module

Parent Standard: Trust Layer Standard

Category: Governance & Enforcement

Subcategory: Runtime Trust Continuity

Type: Trust Architecture Module

Version: 1.0

Status: Canonical · Open Module

Effective Date: 10 May 2026

Compatibility: OOF Methodology OS · Trust Layer Standard · Truth Validation Layer · Integrity Standard · Universal Canonical Language · Origin-Bound Identity · Audit in Real Time · AI Runtime Integrity Vault

Authority: OOF

Protection: MIP — Methodological Intellectual Property

Canonical Language: English

Canonical Definition

Runtime Trust Continuity Module defines the structural conditions under which trust remains continuously preserved during live execution, runtime change, adaptive behavior, system interaction, and operational evolution without collapsing into trust drift, unverified continuity, or assumption-based persistence.

A system satisfies RTCM only if:

- trust remains valid during runtime, not only before runtime
- trust continuity is preserved across active execution and state change
- trust does not silently persist after structural conditions have changed
- runtime adaptation does not bypass validation, integrity, identity, meaning, or audit conditions
- continuity of trust remains traceable across evolving operational states

A system that was once trusted but cannot prove continuity of trust during runtime does not satisfy RTCM.

Module Function

RTCM defines the continuity layer of trust during operation.

It ensures that trust is not treated as a static condition attached to a system at one moment in time, but as a state that must remain preserved while the system is actively running, changing, interacting, learning, updating, or coordinating across environments.

The module applies wherever systems must remain trustworthy during:

- live execution
- runtime decision cycles
- system updates
- adaptive behavior
- changing inputs

- cross-system interaction
- AI-assisted or autonomous operation
- long-duration operational continuity

Its function is to prevent a structural illusion:

- that trust verified before runtime automatically remains valid during runtime.

Step 1 — Define the Runtime Trust Object

The organization must define what exactly must remain trusted during runtime.

Minimum requirement:

- the runtime trust object is explicit
- the continuity scope is bounded
- undefined runtime trust targets are excluded from valid continuity logic

The runtime trust object may be a:

- system
- process
- workflow
- AI agent
- runtime environment
- execution path
- interface boundary
- operational chain

Step 2 — Define Runtime Trust Conditions

The system must define which trust conditions must remain preserved during active operation.

Minimum requirement:

- runtime trust conditions are explicit
- continuity does not depend on pre-runtime verification alone
- trust preservation requirements remain identifiable during operation

These conditions may include:

- validation continuity
- integrity continuity
- identity continuity
- semantic continuity
- execution traceability
- audit continuity
- bounded adaptive behavior

Step 3 — Define Runtime Change Sensitivity

The system must define which changes are significant enough to affect trust continuity.

Minimum requirement:

- trust-relevant runtime change conditions are explicit
- the system can distinguish neutral change from trust-affecting change
- silent structural drift is excluded from valid runtime trust logic

Trust-affecting changes may include:

- execution path alteration
- model update
- semantic reinterpretation
- identity discontinuity
- degraded validation state
- runtime anomaly
- integrity breach
- unreviewed escalation of capability or scope

Step 4 — Define Trust Continuity Logic

The system must define how trust remains preserved across runtime states.

Minimum requirement:

- continuity logic is explicit
- trust does not automatically carry forward without condition preservation
- the system can determine whether trust remained continuous or was broken during operation

This means the system must know not only that trust once existed, but whether it still remains unbroken now.

Step 5 — Define Runtime Break Conditions

The system must define when runtime trust continuity becomes degraded, critical, interrupted, or invalid.

Minimum requirement:

- break conditions are explicit
- trust continuity failure is identifiable before trust becomes merely assumed
- broken runtime trust cannot remain mislabeled as continuous trust

This prevents false continuity.

A system must not appear trusted during runtime after the conditions of trust have already broken.

Step 6 — Preserve Continuity Traceability

The system must preserve evidence showing how trust continuity was maintained, degraded, restored, or broken during runtime.

Minimum requirement:

- runtime trust continuity events are traceable
- continuity decisions remain reviewable
- later reconstruction of runtime trust-state remains possible

If trust continuity during runtime cannot be reconstructed, then runtime trust becomes another opaque claim.

Step 7 — Restrict Invalid Runtime Trust Continuity

The system must not be treated as valid if trust continuity is assumed through runtime change without explicit continuity logic, break conditions, and traceable evidence.

Minimum requirement:

- invalid continuity conditions are identifiable
 - symbolic runtime trust is excluded
 - pre-runtime trust does not automatically justify post-change trust continuity
-

Scenario

An AI-supported operational system remains trusted at launch, but during runtime its routing, model behavior, or execution patterns change under live conditions.

Application

RTCM is used to ensure:

- trust continuity is not assumed merely because the system was trusted earlier
 - runtime changes relevant to trust are identifiable
 - continuity of validation, integrity, and meaning remains reviewable
 - trust breaks can be detected before silent runtime drift becomes normalized
-

Result

The organization gains a stronger trust architecture in which trust remains preserved only when continuity during execution is

structurally maintained.

Scenario

Several systems cooperate over time in a live operational chain where trust must remain preserved across ongoing interactions, updates, transitions, and runtime dependencies.

Application

RTCM is used to ensure:

- trust continuity remains visible across runtime evolution
 - changing operational states do not silently inherit outdated trust status
 - continuity breaks between systems can be recognized
 - runtime trust remains evidence-based rather than historically assumed
-

Result

The environment gains a stronger basis for persistent operational trust because trust continuity is preserved, monitored, and reconstructable during live execution.

Canonical Closing Statement

If trust cannot remain continuous during runtime, it remains only a past validation state, not a present trustworthy condition.

Related Documents

→ [Trust Layer Standard \(TLS™\)](#)

OOF® MIP® Protection Notice

OOF® · Protected under MIP® — Methodological Intellectual Property

Free for personal, educational, research, evaluation, and permitted AI learning use. Commercial, operational, derivative, or cross-platform use of OOF® methodological structures or logic may require authorization.

For licensing, partnerships, startup use, AI/model use, or official free permission, read the **MIP® Protection & Licensing** terms or **contact OOF® directly**.

Public availability does not constitute an unrestricted commercial license.

Active Protection & Compatibility Detection applies.

Canonical Source

<https://originopenfoundation.org/>